算法板子速查

59 个模板 · 2795 行 · A4 双栏

基础模板 2

head 2

STL 算法 2

常用stl算法上 2

常用stl算法下 2

排序 2

归并排序 2

二分 3

二分搜索 3

前缀和 3

一维前缀和 3

二维前缀和 3

差分 3

一维差分 3

二维差分 3

搜索 4

dfs全排列 4

dij最短路径 4

动态规划 4

一维背包 4

图论 5

LCA最短路 5

dij最短路径 5

tarjan 6

所有到剩余点距离最近的点 6

树上换父亲询问全局最大根路径和 7

字符串 7

int与二进制string相互转化 7

字符串哈希 7

kmp 8

字符串转十进制数 8

最长回文子串问题 8

求字符串子序列匹配数 8

马拉车算法 9

trie 10

线段树求字符串最长长度和连续数量 11

数据结构 12

一维树状数组区间修改 12

一维树状数组单点修改 12

优先队列 12

并查集 12

普通树状数组 13

珂朵莉树 13

随机哈希 13

区间加乘和单点查询线段树 14

区间更新线段树 15

无区间更新线段树 16

树状数组异或求和 17

数学 17

gcd求最大公因数 17

余数求和 17

分解质因数 18

容斥原理算区间互质数量 18

快速幂 18

按位异或和 18

杨辉三角 18

构造格雷码 19

模逆元 19

欧拉函数快速幂 19

求二进制的1位数 19

矩阵快速幂 19

线性基求最大异或和之和 20

组合数 20

质数筛 20

输出int128 20

进制转化 20

大数加减乘除 21

几何 22

叉乘公式 22

线段在圆中的长度函数 22

模拟 23

德州扑克 23

对拍 23

生成随机数 23

基础模板 1 个模板

head 17 行

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
void solve(){

}
int main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    int t = 1;
    //cin >> t;
    while (t--){
        solve();
    }
    return 0;
}
```

STL 算法 2 个模板

常用stl算法上 31 行

```cpp
#include<bits/stdc++.h>
using namespace std;
//find
//reverse
//unique
//erase
//sort
//nth_element
//binary_search
bool sort_way(int a,int b){return a<b;}
int main(){
    int n,tar;cin>>n>>tar;
    vector<int> nums(n);
    for(int i=0;i<n;i++)cin>>nums[i];
    auto it = find(nums.begin(),nums.end(),tar);//find: 顺序查找
    int pos = it-nums.begin();
    reverse(nums.begin(),nums.end());//reverse: 反转数组
    //unique: 移除相邻重复元素
    //ps: 不改变容器大小，只是把不重复的元素向前移动，所以要使用erase或者pop_back()，最好在之前sort一遍
    auto last = unique(nums.begin(),nums.end());
    nums.erase(last,nums.end()); //erase: 删除这一部分的元素
    nums.erase(nums.begin()+2, nums.begin()+3);// 删除中间一部分，前闭后开
    sort(nums.begin(),nums.end(),sort_way);//stable_sort: 可以保证相等元素的相对顺序，但用处较少，一般使用普通sort
    it = nums.begin()+3;//第四个位置
    nth_element(nums.begin(),it,nums.end(),sort_way);//nth_element: 不做排序，只找到第n小的数,时间复杂度位O(n)
    cout<<*it<<endl;
    //binary_search要求必须按照规则排序，否则会报错
    bool bs = binary_search(nums.begin(),nums.end(),tar,sort_way);//binary_search: 不返回迭代器，只返回有没有找到

    for(auto num:nums)cout<<num<<' ';
}
```

常用stl算法下 40 行

```cpp
#include<bits/stdc++.h>
using namespace std;
bool cmp(int a,int b){return a<b;}
//merge
//lower_bound
//upper_bound
//next_permutation
//partial_sum
int main(){
    vector<int> v1 = {1,3,5};
    vector<int> v2 = {2,4,6};
    vector<int> v3(v1.size()+v2.size());
    //merge添加的排序规则要和v1v2一样
    merge(v1.begin(),v1.end(),v2.begin(),v2.end(),v3.begin(),cmp);//merge:两个已排序序列有序合并到第三个序列的插入迭代器上
    auto it1 = lower_bound(v1.begin(),v1.end(),3);//lower_bound: 使用二分查找搜索第一个不小于x的值
    auto it2 = upper_bound(v1.begin(),v1.end(),3);//upper_bound: 使用二分查找搜索第一个大于x的值
    //返回迭代器
    int pos1 = it1-v1.begin(),pos2 = it2-v1.begin();
    cout<<pos1<<' '<<pos2<<endl;
    vector<int> v = v1;
    do{
        for(int n:v)cout<<n<<' ';
        cout<<'\n';
        //上一个排列使用prev_permutation
    }while(next_permutation(v.begin(),v.end()));//next_permutation: 将当前排列更改为全排列中的下一个排列
    // for(auto n:v3)cout<<n<<' ';
    vector<int> input = {1,2,3,4,5};
    vector<int> res(input.size());
    partial_sum(input.begin(),input.end(),res.begin());//partial_sum: 统计前缀和
    for(int i=0;i<(int)res.size();i++)cout<<res[i]<<' ';
    int l=1,r=2;
    int mx = ceil((l+r-1)/r);//向上取整
    int p = 0;
    vector<int>b;
     multiset<int> ms(b.begin(), b.end());//查找删除都是logn
     auto it = ms.lower_bound(1 - p);
     int d = *it;
     ms.erase(it);
    return 0;
}
```

排序 1 个模板

归并排序 24 行

```cpp
#include<bits/stdc++.h>
using namespace std;
//归并排序求逆序对数量
int a[500005],c[500005];
long long ans = 0;
void to_cnt(int l,int r){
    if(l==r)return;
    int mid = (l+r)/2,i = l,j = mid+1,k = l;
    to_cnt(l,mid);to_cnt(mid+1,r);
    while(i<=mid&&j<=r){
        if(a[i]<=a[j])c[k++] = a[i++];
        else {c[k++] = a[j++];ans += mid-i+1;}
    }
    while(i<=mid) c[k++] = a[i++];
    while(j<=r) c[k++] = a[j++];
    for(int m = l;m <= r;m++)a[m] = c[m];
}
int main(){
    int n;cin>>n;
    for(int i=1;i<=n;i++)cin>>a[i];
    to_cnt(1,n);
    cout<<ans<<endl;
    return 0;
}
```

二分 1 个模板

二分搜索 48 行

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll INF = (ll)1e18;
bool check(vector<int>& nums,int mid,int tar){
    return nums[mid]<=tar;
}
int bs(vector<int> nums,int tar){
    int l = 0, r = (int)nums.size() - 1;
    while(l<r){
        int mid = (l+r+1)/2;
        if(check(nums,mid,tar))l=mid;
        else r=mid-1;
    }
    return l;
}

bool check_ll(ll x){
    if(x==0)return true;
    return x<=INF/x;
}
ll bs_ll(ll l,ll r){
    while(l<r){
        ll mid = (l+r+1)/2;
        if(check_ll(mid))l=mid;
        else r=mid-1;
    }
    return l;
}

void solve(){
    vector<int> nums(10);
    for(int i=0;i<10;i++)nums[i]=i;
    cout<<bs(nums,5)<<"\n";
    cout<<bs_ll(0,INF)<<"\n";
}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    int t = 1;
    //cin >> t;
    while (t--){
        solve();
    }
    return 0;
}
```

前缀和 2 个模板

一维前缀和 15 行

```
#include<bits/stdc++.h>
using namespace std;
vector<int> prefix_sum(vector<int> nums){
    int n = nums.size();
    vector<int> pre(n+1,0);
    for(int i=1;i<=n;i++)pre[i]=pre[i-1]+nums[i-1];
    return pre;
}
int main(){
    vector<int> nums(10);
    for(int i=0;i<10;i++)nums[i]=i+1;
    vector<int> pre = prefix_sum(nums);
    for(int i=0;i<=10;i++)cout<<pre[i]<<endl;
    return 0;
}
```

二维前缀和 23 行

```
#include<bits/stdc++.h>
using namespace std;
vector<vector<int>> td_prefix_sum(vector<vector<int> > nums){
    int n = nums.size();
    vector<vector<int>> pre(n+1,vector<int>(n+1,0));
    for(int i=1;i<=n;i++)
        for(int j=1;j<=n;j++)
            pre[i][j]=pre[i-1][j]+pre[i][j-1]-pre[i-1][j-1]+nums[i-1
  ][j-1];
    return pre;
}
int main(){
    int n;
    cin>>n;
    vector<vector<int>> nums(n,vector<int>(n,1));
    vector<vector<int>> pre = td_prefix_sum(nums);
    for(int i=0;i<=n;i++){
        for(int j=0;j<=n;j++){
            cout<<pre[i][j]<<' ';
        }
        cout<<endl;
    }
    return 0;
}
```

差分 2 个模板

一维差分 21 行

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    int len,t;
    cin>>len>>t;
    vector<int> nums(len);
    for(int i=0;i<len;i++)cin>>nums[i];
    vector<int> diff(len+2,0);
    while(t--){
        int l,r,c;
        cin>>l>>r>>c;
        diff[l]+=c;
        diff[r+1]-=c;
    }
    for(int i=1;i<=len;i++){
        diff[i]+=diff[i-1];
        nums[i-1]+=diff[i];
    }
    for(int i=0;i<len;i++)cout<<nums[i]<<endl;
    return 0;
}
```

二维差分 31 行

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    int m,n;
    cin>>m>>n;
    vector<vector<int>> nums(m,vector<int>(n));
    for(int i=0;i<m;i++) for(int j=0;j<n;j++) cin>>nums[i][j];

    vector<vector<int>> diff(m+2,vector<int>(n+2,0));
    int t; cin>>t;
    while(t--){
        int x1,y1,x2,y2,c;
        cin>>x1>>y1>>x2>>y2>>c;
        diff[x1][y1]+=c;
        diff[x1][y2+1]-=c;
        diff[x2+1][y1]-=c;
        diff[x2+1][y2+1]+=c;
    }
    for(int i=1;i<=m;i++){
        for(int j=1;j<=n;j++){
            diff[i][j]+=diff[i-1][j]+diff[i][j-1]-diff[i-1][j-1];
            nums[i-1][j-1]+=diff[i][j];
        }
    }
    for(int i=0;i<m;i++){
        for(int j=0;j<n;j++){
            cout<<nums[i][j];
        }
        cout<<endl;
    }
    return 0;
}
```

搜索 2 个模板

dfs全排列

45 行

```cpp
#include<bits/stdc++.h>
using namespace std;
int n;
vector<bool> bl;
vector<vector<int>> res;
vector<int> tmp;
void dfs(){
    if(tmp.size()==n){
        res.push_back(tmp);
        return ;
    }
    for(int i=1;i<=n;i++){
        if(bl[i]){
            bl[i]=false; tmp.push_back(i);
            dfs();
            tmp.pop_back(); bl[i]=true;
        }
    }
    return ;
}
void np_dfs(vector<int> a){
    while(next_permutation(a.begin(), a.end())){
        for(int i=0;i<(int)a.size();i++)cout<<a[i]<<' ';
        cout<<endl;
    }
    return ;
}
int main(){
    ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
    // cin>>n;
    // bl.resize(n+1,true);
    // dfs();
    // int len = res.size();
    // for(int i=0;i<len;i++){
    //     for(int j=0;j<n;j++){
    //         cout<<res[i][j]<<' ';
    //     }
    //     cout<<"\n";
    // }
    cin>>n;
    vector<int> nums(n);
    for(int i=0;i<n;i++)nums[i]=i;
    np_dfs(nums);
    return 0;
}
```

dij最短路径

53 行

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
int n,m,q;
vector<vector<pair<int,int>>> gp;
vector<int> dis;
void dij(){
    dis.assign(n+1,INT_MAX);
    dis[1] = 0;
    priority_queue<pair<int,int>,vector<pair<int,int>>,greater<>> qu
  e;
    que.emplace(0,1);
    while(!que.empty()){
        auto [d,x] = que.top();que.pop();
        if(dis[x]==d){
            for(const auto& [y,v]:gp[x]){
                int newdis = d+v;
                if(newdis<dis[y]){
                    dis[y] = newdis;
                    que.emplace(newdis,y);
                }
            }
        }
    }
}
void solve(){
    cin>>n>>m>>q;
    gp.resize(n+1);
    while(m--){
        int u,v,w;
        cin>>u>>v>>w;
        gp[u].push_back({v,w});
        gp[v].push_back({u,w});
    }
    dij();
    ll res = 0;
    while(q--){
        int a;cin>>a;
        res=dis[a];
        cout<<res<<endl;
    }

}
int main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    int t = 1;
    //cin >> t;
    while (t--){
        solve();
    }
    return 0;
}
```

动态规划 1 个模板

一维背包

19 行

```cpp
#include<bits/stdc++.h>
using namespace std;
vector<int> to_b(vector<int> w,vector<int> v,int W){
    int N = w.size();
    vector<int> b(W+1,0);
    for(int i=0;i<N;i++){
        for(int j=W;j>=w[i];j--) b[j] = max(b[j],b[j-w[i]]+v[i]);
    }
    return b;
}
//背包板子
int main(){
    int N,W;cin>>N>>W;
    vector<int> w(N),v(N);
    for(int i=0;i<N;i++)cin>>w[i]>>v[i];
    vector<int> b = to_b(w,v,W);
    cout<<b.back()<<endl;
    return 0;
}
```

图论 5 个模板

dij最短路径

53 行

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
int n,m,q;
vector<vector<pair<int,int>>> gp;
vector<int> dis;
void dij(){
    dis.assign(n+1,INT_MAX);
    dis[1] = 0;
    priority_queue<pair<int,int>,vector<pair<int,int>>,greater<>> qu
  e;
    que.emplace(0,1);
    while(!que.empty()){
        auto [d,x] = que.top();que.pop();
        if(dis[x]==d){
            for(const auto& [y,v]:gp[x]){
                int newdis = d+v;
                if(newdis<dis[y]){
                    dis[y] = newdis;
                    que.emplace(newdis,y);
                }
            }
        }
    }
}
void solve(){
    cin>>n>>m>>q;
    gp.resize(n+1);
    while(m--){
        int u,v,w;
        cin>>u>>v>>w;
        gp[u].push_back({v,w});
        gp[v].push_back({u,w});
    }
    dij();
    ll res = 0;
    while(q--){
        int a;cin>>a;
        res=dis[a];
        cout<<res<<endl;
    }

}
int main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    int t = 1;
    //cin >> t;
    while (t--){
        solve();
    }
    return 0;
}
```

LCA最短路

89 行

```cpp
#include <algorithm>
#include <iostream>
#include <vector>
using namespace std;
//N个点 N-1条边 q次询问查找两个点之间的最短路长度
int n;
vector<vector<int>> graph;
int _u;
int _v;
int q;

vector<int> h;

vector<vector<int>> st = {{}};
vector<int> pos;

int Min(int x, int y) {
    if (h[x] < h[y]) {
        return x;
    }
    return y;
}

void Dfs(int x, int fa) {
    pos[x] = st[0].size();
    st[0].push_back(x);
    for (const auto& y : graph[x]) {
        if (y != fa) {
            h[y] = h[x] + 1;
            Dfs(y, x);
            st[0].push_back(x);
        }
    }
}

void InitST() {
    st.resize(__lg(st[0].size()) + 1);
    for (int i = 1; i < st.size(); i++) {
        st[i].resize(st[0].size() - (1 << i) + 1);
        for (int j = 0; j < st[i].size(); j++) {
            st[i][j] = Min(st[i - 1][j], st[i - 1][j + (1 << (i - 1)
  )]);
        }
    }
}

int Mini(int l, int r) {
    if (l > r) {
        swap(l, r);
    }
    int t = __lg(r - l + 1);
    return Min(st[t][l], st[t][r - (1 << t) + 1]);
}

int Lca(int a, int b) {
    return Mini(pos[a], pos[b]);
}

int Dis(int a, int b) {
    return h[a] + h[b] - h[Lca(a, b)] * 2;
}

void Solve() {
    cin >> n;
    graph.resize(n + 1);
    for (int i = 1; i < n; i++) {
        int u, v;
        cin >> u >> v;
        graph[u].push_back(v);
        graph[v].push_back(u);
    }
    cin >> _u >> _v >> q;
    h.resize(n + 1);
    pos.resize(n + 1);
    Dfs(1, 0);
    InitST();
    while (q--) {
        int x, y;
        cin >> x >> y;
        cout << min({Dis(x, y), Dis(x, _u) + Dis(_v, y), Dis(x, _v)
  + Dis(_u, y)}) <<
             '\n';
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    Solve();
}
// 64 位输出请用 printf("%lld")
```

tarjan

74 行

```cpp
//题目https://www.nowcoder.com/practice/d53b2057ae8a4eaf8155447f6b709361?channelPut=tracker2
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const int MOD = 1e9+7;
int n;
vector<int> w;
vector<vector<int>> g;
//对于构成回路，也就是首尾相连的情况下，只要找到回路里面最小的那个点的权值
//对于无法构成回路的独立点必须单独计费
//理论上来说 每一个回路的最小点的数量相乘应该就是方案数
vector<int> dfn,low,stk;
vector<bool> in;
int dfnCc = 1;
ll tot = 0;
ll fangan = 1;
void dfs(int x){
    dfn[x] = low[x] = dfnCc;
    dfnCc++;
    stk.push_back(x);
    in[x] = true;
    for(const auto& y:g[x]){
        if(in[y]){
            low[x] = min(low[x],dfn[y]);
            continue;
        }
        if(!dfn[y]){
            dfs(y);
            low[x] = min(low[x],low[y]);
        }
    }
    int mini = INT_MAX;
    int cnt = 0;
    if(dfn[x]==low[x]){
        int z = 0;
        while(z!=x){
            z = stk.back();stk.pop_back();
            in[z] = false;
            if(w[z]<mini){mini = w[z];cnt = 1;}
            else if(w[z]==mini)cnt++;
        }
        fangan = fangan*cnt%MOD;
        tot+=mini;
    }
}
void solve(){
    cin>>n;
    w.resize(n+1);
    g.resize(n+1);
    dfn.resize(n+1);
    low.resize(n+1);
    in.resize(n+1);
    for(int i=1;i<=n;i++)cin>>w[i];
    int m;cin>>m;
    for(int i=0;i<m;i++){
        int x,y;cin>>x>>y;
        g[x].push_back(y);
    }
    for(int i=1;i<=n;i++){
        if(!dfn[i])dfs(i);
    }
    cout<<tot<<' '<<fangan;
}
int main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    ll t = 1;
    // cin >> t;
    while (t--){
        solve();
    }
    return 0;
}
```

所有到剩余点距离最近的点

56 行

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const int N = 5e4+10;
//给定一个图，然后找出所有到剩余点距离最近的点
vector<vector<int>> g;
ll cnt[N],sum[N],dep[N];
ll mn,m,n;
void dfs1(int p,int fa){
    dep[p] = dep[fa]+1;
    cnt[p] = 1;
    for(int i=0;i<(int)g[p].size();i++){
        if(g[p][i]!=fa){dfs1(g[p][i],p);cnt[p]+=cnt[g[p][i]];}
    }
}
void dfs2(int p,int fa){
    for(int i=0;i<(int)g[p].size();i++)
        if(g[p][i]!=fa){
            sum[g[p][i]] = sum[p]-cnt[g[p][i]]+(n-cnt[g[p][i]]);
            mn = min(mn,sum[g[p][i]]);
            dfs2(g[p][i],p);
        }
}
void solve(){
    cin>>n>>m;
    g.resize(n+1);
    map<int,map<int,int>> mp;
    for(int i=1;i<=m;i++){
        int u,v;cin>>u>>v;
        if(mp[u][v]==0){
            g[u].push_back(v);
            g[v].push_back(u);
            mp[u][v] = 1;mp[v][u] = 1;
        }
    }
    dep[0] = -1;
    dfs1(1,0);
    for(int i=1;i<=n;i++)sum[1]+=dep[i];
    mn = sum[1];
    dfs2(1,0);
    for(int i=1;i<=n;i++){
        if(sum[i]==mn)cout<<i<<' ';
    }
    cout<<endl;
}
int main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    int t = 1;
    //cin >> t;
    while (t--){
        solve();
    }
    return 0;
}
```

树上换父亲询问全局最大根路径和

109 行

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const int N = 2e5 + 5;

ll rnk[N];
ll a[N], sum[N];
int p[N];
vector<int> e[N];

struct SegT {
    ll mx[N*4];
    ll tag[N*4];

    void up(int i) {
        mx[i] = max(mx[i*2], mx[i*2+1]);
    }

    void lazy(int i,ll v) {
        tag[i] += v;
        mx[i] += v;
    }
    void down(int i) {
        if (tag[i]) {
            lazy(i*2,tag[i]);
            lazy(i*2+1,tag[i]);
            tag[i] = 0;
        }
    }
    void build(int i,int l,int r) {
        if (l == r) {
            mx[i] = rnk[l];
        } else {
            int mid = (l + r) / 2;
            build(i*2,l,mid);
            build(i*2+1,mid+1,r);
            up(i);
        }
    }
    void add(int i,int l,int r,int jl,int jr,ll v) {
        if (jl <=l && r <= jr) {
            lazy(i,v);
        } else {
            int mid = (l + r) / 2;
            down(i);
            if (jl <= mid) add(i*2,l,mid,jl,jr,v);
            if (jr > mid) add(i*2+1,mid+1,r,jl,jr,v);
            up(i);
        }
    }
    ll query(int i,int l,int r,int jl,int jr) {
        if (jl <= l && r<= jr) {
            return mx[i];
        } else {
            ll res = -1e18;
            int mid = (l + r) / 2;
            down(i);
            if (jl <= mid) res = max(res, query(i*2,l,mid,jl,jr));
            if (jr > mid)res = max(res, query(i*2+1,mid+1,r,jl,jr));
            up(i);
            return res;
        }
    }
}T;

int n, dfncnt;
int L[N];
int R[N];

void dfs(int u) {
    L[u] = ++dfncnt;
    sum[u] += a[u];
    rnk[dfncnt] = sum[u];
    for (auto v: e[u]) {
        sum[v] = sum[u];
        dfs(v);
    }
    R[u] = dfncnt;
}

void solve() {
    int q;
    cin >> n >> q;
    for (int i = 1; i <= n; i++) cin >> a[i];
    for (int i = 2; i <= n; i++) {
        cin >> p[i];
        e[p[i]].push_back(i);
    }
    dfs(1);
    T.build(1,1,n);
    for (int i= 1; i <= q; i++) {
        int x, y;
        cin >> y >> x;
        int z = p[y];
        ll delta = sum[x]-sum[z];
        T.add(1,1,dfncnt,L[y],R[y],delta);
        cout << T.query(1,1,dfncnt,1,dfncnt) << "\n";
        T.add(1,1,dfncnt,L[y],R[y],-delta);
    }
}

int main() {
    ios::sync_with_stdio(0); cin.tie(0); cout.tie(0);
    int T = 1;
    // cin >> T;
    while (T--) solve();
}
// 64 位输出请用 printf("%lld")
```

字符串 9 个模板

字符串哈希

26 行

```cpp
#include<bits/stdc++.h>
using namespace std;
// 如果两字符串相等，返回true

constexpr int M = 1e9 + 7;
constexpr int B = 233;

using ll = long long;

int get_hash(const string& s) {
  int res = 0;
  for (int i = 0; i < s.size(); ++i) {
    res = ((ll)res * B + s[i]) % M;
  }
  return res;
}

bool cmp(const string& s, const string& t) {
  return get_hash(s) == get_hash(t);
}
int main(){
    string s1,s2;cin>>s1>>s2;
    if(cmp(s1,s2))cout<<"Yes"<<endl;
    else cout<<"No"<<endl;
    return 0;
}
```

int与二进制string相互转化

32 行

```cpp
#include<bits/stdc++.h>
using namespace std;
//贪心构造
string int_to_bit(int n){
    if(n==0)return "0";
    string s;
    while(n>0){
        s.push_back((char)(n%2+'0'));
        n/=2;
    }
    reverse(s.begin(),s.end());
    return s;
}
int bit_to_int(string s){
    int len = s.length(),pos = -1;
    for(int i=0;i<len;i++){
        if(s[i]=='1'){pos = i;break;}
    }
    if(pos==-1||len==0)return 0;
    else if(pos==len- 1)return 1;
    int ans = 1;
    for(int i=pos+1;i<len;i++){
        ans*=2;
        if(s[i]=='1')ans++;
    }
    return ans;
}
int main(){
    int n;cin>>n;

    return 0;
}
```

kmp

43 行

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
vector<int> kmp(string s1,string s2){
    vector<int> kmps;
    int len2 = s2.size();
    kmps.resize(len2);
    int j = 0;
    for(int i=1;i<len2;i++){
        while(j&&s2[i]!=s2[j]) j = kmps[j-1];
        if(s2[i] == s2[j])j++;
        kmps[i] = j;
    }
    j = 0;
    vector<int> ans;
    for(int i=0;i<(int)s1.size();i++){
        while(j&&s1[i]!=s2[j])j = kmps[j-1];
        if(s1[i]==s2[j]){
            j++;
            if(j==len2){
                ans.push_back(i+2-len2);
                j = kmps[j-1];
            }
        }
    }
    return ans;
}
void solve(){
    string s1,s2;cin>>s1>>s2;
    vector<int> ans = kmp(s1,s2);
    for(int i=0;i<(int)ans.size();i++)cout<<ans[i]<<' ';
}
int main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    int t = 1;
    //cin >> t;
    while (t--){
        solve();
    }
    return 0;
}
```

字符串转十进制数

20 行

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
int strto_num(string s){
        int pos = -1,len = s.length();
        for(int i=0;i<len;i++)if(s[i]!='0'){pos = i;break;}
        if(pos==-1)return 0;
        int ans = 0;
        for(int i=pos;i<len;i++){
            int t = (int)(s[i]-'0');
            ans = ans*10+t;
        }
        return ans;
    }
int main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    return 0;
}
```

求字符串子序列匹配数

21 行

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int numDistinct(string s, string t) {
        int n = s.length(),m = t.length();
        if(n<m)return 0;
        vector<vector<unsigned long long>> dp(n+1,vector<unsigned lo
  ng long>(m+1,0));
        for(int i=0;i<=n;i++) dp[i][m] = 1;
        for(int i=n-1;i>=0;i--){
            char a = s[i];
            for(int j=m-1;j>=0;j--){
                char b = t[j];
                if(a == b)dp[i][j] = dp[i+1][j]+dp[i+1][j+1];
                else dp[i][j] = dp[i+1][j];
            }
        }
        return dp[0][0];
    }
};
```

最长回文子串问题

44 行

```
#include<bits/stdc++.h>
using namespace std;
typedef unsigned long long ULL;
const int P = 131;
const int N = 1000010;
//字符串哈希+二分
ULL h1[N],h2[N],p[N];
char s[N];
ULL get_hash(ULL h[],int l,int r){
    return h[r] - h[l-1]*p[r-l+1];
}
int main(){
    string si;cin>>si;
    int n = si.length();
    for(int i=1;i<=n;i++)s[i] = si[i-1];
    p[0] = 1;
    for(int i=1;i<=n;i++){
        p[i] = p[i-1]*P;
        h1[i] = h1[i-1]*P+s[i];
        h2[i] = h2[i-1]*P+s[n-i+1];
    }
    int maxlen = 0;
    for(int i=1;i<=n;i++){
        int l = 0,r = min(i-1,n-i);
        while(l<r){
            int mid = (l+r+1)>>1;
            if(get_hash(h1,i-mid,i-1)==get_hash(h2,n-(i+mid)+1,n-(i+
  1)+1)){
                l = mid;
            }
            else r = mid-1;
        }
        maxlen = max(maxlen,1+2*l);
        l = 0,r = min(i,n-i);
        while(l<r){
            int mid = (l+r+1)>>1;
            if(get_hash(h1,i-mid+1,i)==get_hash(h2,n-(i+mid)+1,n-(i+
  1)+1)){
                l = mid;
            }else r = mid-1;
        }
        maxlen = max(maxlen,2*l);
    }
    cout<<maxlen<<endl;
    return 0;
}
```

马拉车算法

49 行

```cpp
#include<bits/stdc++.h>
using namespace std;
//求最长回文子串O(n)马拉车算法
string preprocess(const string &s){
    string t = "^.";
    for(auto c : s){
        t.push_back(c);
        t.push_back('.');
    }
    t.push_back('$');
    return t;
}
vector<int> manacher(const string &s){
    string t = preprocess(s);
    int n = t.size();
    int mid = -1, mr = 0;
    vector<int> p(n);
    for(int i = 0; i < n; i++){
        if(i < mr) p[i] = min(p[2 * mid - i], mr - i);
        else p[i] = 1;
        while(t[i - p[i]] == t[i + p[i]]) p[i]++;
        if(i + p[i] > mr){
            mid = i;
            mr = i + p[i];
        }
    }
    return p;
}
//最长回文子串长度
int longestPalindrome(const string &s){
    vector<int> p = manacher(s);
    return *max_element(p.begin(), p.end()) - 1;
}
void solve(){
    string s;
    cin >> s;
    cout << longestPalindrome(s) << endl;
}
int main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    int t = 1;
    //cin >> t;
    while (t--){
        solve();
    }
    return 0;
}
```

trie

95 行

```cpp
#include <bits/stdc++.h>
using namespace std;
const int N = 200000 + 10;
const int CNT = 36;
int trie[N][CNT];
int pass[N], endd[N];// pass: 当前记住的句子经过该节点的数量, endd: 该节点是否为某个记住句子的结尾
int cnt;
inline int get_path(char c) {
    if (c >= 'a' && c <= 'z') return c - 'a';
    return c - '0' + 26;          // 数字映射到 26 ~ 35
}
void build(){
    for(int i = 1; i <= cnt; i++){
        fill(trie[i], trie[i] + CNT, 0);
        pass[i] = 0;
        endd[i] = 0;
    }
    cnt = 1;
}
// 判断一个句子当前是否被记住
bool exists(string str){
    int head = 1;
    for(char c : str){
        int p = get_path(c);
        if(trie[head][p] == 0)
            return false;
        head = trie[head][p];
    }
    return endd[head] > 0;
}
// 插入一个新的句子
void insert(string str){
    int head = 1;
    pass[head]++;
    for(char c : str){
        int p = get_path(c);
        if(trie[head][p] == 0)
            trie[head][p] = ++cnt;
        head = trie[head][p];
        pass[head]++;
    }
    endd[head] = 1;
}
// 删除一个当前记住的句子
void erase(string str){
    int head = 1;
    pass[head]--;
    for(char c : str){
        int p = get_path(c);
        int nxt = trie[head][p];
        pass[nxt]--;
        head = nxt;
    }
    endd[head] = 0; // 不再记住
}
// 查询当前记住的句子中以 prefix 为前缀的句子数量
int prefix_num(string prefix){
    int head = 1;
    for(char c : prefix){
        int p = get_path(c);
        if(trie[head][p] == 0)
            return 0;
        head = trie[head][p];
    }
    return pass[head];
}
int main() {
    ios::sync_with_stdio(0); cin.tie(0); cout.tie(0);
    int n; cin >> n;
    build();
    unordered_map<string, int> mp; // 当前记住的每个句子的出现次数
    mp.reserve(n * 2);
    while(n--){
        int op; string str; cin >> op >> str;
        if(op == 1) {
            if(!exists(str)){
                insert(str);
                mp[str] = 1;  // 重新记忆，计数从1开始
            }
            else
                mp[str]++; // 已经记住，累加
            cout << mp[str] << endl;
        }
        else if(op == 2)
            cout << (prefix_num(str) > 0 ? "YES" : "NO") << endl;
        else{
            if(exists(str)){
                erase(str);
                mp.erase(str); // 忘记该句及其出现次数
            }
        }
    }

    return 0;
}
```

线段树求字符串最长长度和连续数量

178 行

```
#include<bits/stdc++.h>
using namespace std;
struct Nodes{
    int len;
    int pre[3],suf[3],cnt[3],mx[3];
    Nodes(int c = -1):len(c==-1? 0:1){
        memset(pre,0,sizeof pre);
        memset(suf,0,sizeof suf);
        memset(cnt,0,sizeof cnt);
        memset(mx,0,sizeof mx);
        if(c>=0)pre[c] = suf[c] = cnt[c] = mx[c] = 1;
    }
};
class LazySegmentTree {
    using T = Nodes;
    using F = int;
    const F TODO_INIT = 0;
    struct Node{
        T val;
        F todo;
    };
    int n;
    vector<Node> tree;

    T merge_val(const T& a, const T& b) const {
        T res;
        res.len = a.len+b.len;
        for(int i=0;i<3;i++){
            res.pre[i]=(a.pre[i]==a.len)?a.len+b.pre[i]:a.pre[i];
            res.suf[i]=(b.suf[i]==b.len)?b.len+a.suf[i]:b.suf[i];
            res.cnt[i]=a.cnt[i]+b.cnt[i]-(a.suf[i]>0&&b.pre[i]>0?1:0
  );
            res.mx[i]=max({a.mx[i],b.mx[i],a.suf[i]+b.pre[i]});
        }
        return res;
    }

    F merge_todo(const F& a, const F& b) const {
       return (a+b)%3;
    }

    void apply(int node, int l, int r, F todo) {
        T& cur = tree[node].val;
        int npre[3],nsuf[3],ncnt[3],nmx[3];
        for(int i=0;i<3;i++){
            int nc = (i+todo)%3;
            npre[nc] = cur.pre[i];
            nsuf[nc] = cur.suf[i];
            ncnt[nc] = cur.cnt[i];
            nmx[nc] = cur.mx[i];
        }
        for(int i=0;i<3;i++){
            cur.pre[i] = npre[i];
            cur.suf[i] = nsuf[i];
            cur.cnt[i] = ncnt[i];
            cur.mx[i] = nmx[i];
        }
        tree[node].todo = merge_todo(todo,tree[node].todo);
    }

    void spread(int node, int l, int r) {
        Node& cur = tree[node];
        F todo = cur.todo;
        if (todo == TODO_INIT) return;
        int m = (l + r) >> 1;
        apply(node * 2, l, m, todo);
        apply(node * 2 + 1, m + 1, r, todo);
        cur.todo = TODO_INIT;
    }

    void maintain(int node) {
        tree[node].val = merge_val(tree[node * 2].val, tree[node * 2
   + 1].val);
    }

    void build(const vector<T>& a, int node, int l, int r) {
        Node& cur = tree[node];
        cur.todo = TODO_INIT;
        if (l == r) {
            cur.val = a[l];
            return;
        }
        int m = (l + r) >> 1;
        build(a, node * 2, l, m);
        build(a, node * 2 + 1, m + 1, r);
        maintain(node);
    }

    void update(int node, int l, int r, int ql, int qr, F f) {
        if (ql <= l && r <= qr) {
            apply(node, l, r, f);
            return;
        }
        spread(node, l, r);
        int m = (l + r) >> 1;
        if (ql <= m) update(node * 2, l, m, ql, qr, f);
        if (qr > m) update(node * 2 + 1, m + 1, r, ql, qr, f);
        maintain(node);
    }

    T query(int node, int l, int r, int ql, int qr) {
        if (ql <= l && r <= qr) return tree[node].val;
        spread(node, l, r);
        int m = (l + r) >> 1;
        if (qr <= m) return query(node * 2, l, m, ql, qr);
        if (ql > m) return query(node * 2 + 1, m + 1, r, ql, qr);
        T l_res = query(node * 2, l, m, ql, qr);
        T r_res = query(node * 2 + 1, m + 1, r, ql, qr);
        return merge_val(l_res, r_res);
    }

public:

    LazySegmentTree(int n, T init_val = 0) : LazySegmentTree(vector<
  T>(n, init_val)) {}

    LazySegmentTree(const vector<T>& a) : n(a.size()), tree(2 << bit
  _width(a.size() - 1)) {
        build(a, 1, 0, n - 1);
    }//c++20

    void update(int ql, int qr, F f) {
        update(1, 0, n - 1, ql, qr, f);
    }

    T query(int ql, int qr) {
        return query(1, 0, n - 1, ql, qr);
    }
    int to_ans()const{
        return tree[1].val.cnt[0];
    }
    int to_max()const{
        return tree[1].val.mx[0];
    }
};

void solve_cnt(){
    int n,q;cin>>n>>q;
    string s;cin>>s;
    vector<Nodes> a(n);
    for(int i=0;i<n;i++)a[i] = Nodes(s[i]-'A');
    LazySegmentTree seg(a);
    while(q--){
        int op;cin>>op;
        if(op==1){
            int l,r;cin>>l>>r;
            seg.update(l-1,r-1,1);
        }else{
            cout<<seg.to_ans()<<endl;
        }
    }
}

void solve_max(){
    int n,q;cin>>n>>q;
    string s;cin>>s;
    vector<Nodes> a(n);
    for(int i=0;i<n;i++)a[i] = Nodes(s[i]-'A');
    LazySegmentTree seg(a);
    while(q--){
        int op;cin>>op;
        if(op==1){
            int l,r;cin>>l>>r;
            seg.update(l-1,r-1,1);
        }else{
            cout<<seg.to_max()<<endl;
        }
    }
}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    int t = 1;
    //cin >> t;
    while (t--){
        //solve_cnt();
        solve_max();
    }
    return 0;
}
```

数据结构 11 个模板

一维树状数组区间修改 47 行

```
//一维树状数组板子
#include<bits/stdc++.h>
using namespace std;
const int MAXN = 1e6+7;
int t1[MAXN], t2[MAXN], n;

int lowbit(int x) { return x & (-x); }

void add(int k, int v) {
  int v1 = k * v;
  while (k <= n) {
    t1[k] += v, t2[k] += v1;
    k += lowbit(k);
  }
}

int getsum(int *t, int k) {
  int ret = 0;
  while (k) {
    ret += t[k];
    k -= lowbit(k);
  }
  return ret;
}

void add1(int l, int r, int v) {
  add(l, v), add(r + 1, -v);
}

long long getsum1(int l, int r) {
  return (r + 1ll) * getsum(t1, r) - 1ll * l * getsum(t1, l - 1) -
         (getsum(t2, r) - getsum(t2, l - 1));
}
int main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin>>n;
    vector<int> a(n+1);
    for(int i=1;i<=n;i++)cin>>a[i];
    memset(t1,0,sizeof(t1));
    memset(t2,0,sizeof(t2));
    for(int i=1;i<=n;i++)add1(i,i,a[i]);
    int l,r,x;cin>>l>>r>>x;
    add1(l,r,x);
    long long ans = getsum1(l,r);
    cout<<ans<<endl;
}
```

优先队列 24 行

```
#include<bits/stdc++.h>
using namespace std;
//结构体自定义
struct Node{
    int x,y;
};
struct cmp{
    bool operator()(const Node &a, const Node &b) const{
        return a.y < b.y;
    }
};
int main(){
    priority_queue<int>bpq;//最大堆，每次入队自动将最大值放在队头
    priority_queue<int,vector<int>,greater<int>> mpq;//最小堆，和最大堆相反
    priority_queue<Node, vector<Node>, cmp> pq;
    for(int i=1;i<=10;i++){
        bpq.push(i);
        mpq.push(i);
    }
    while(!bpq.empty()){cout<<bpq.top()<<' ';bpq.pop();}
    cout<<endl;
    while(!mpq.empty()){cout<<mpq.top()<<' ';mpq.pop();}
    return 0;
}
```

一维树状数组单点修改 50 行

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
//构建可以计算前缀的树状数组
const ll MAXN = 2e6+10;
vector<ll> c(MAXN+1),b(MAXN+1);
ll lowbit(ll x){return x & -x;}
ll getsum(ll x,vector<ll>&c){
    ll ans = 0;
    while(x>0){
        ans+=c[x];
        x-=lowbit(x);
    }
    return ans;
}
void add(ll x,ll k,vector<ll>&c){
    while(x<=MAXN){
        c[x]+=k;
        x+=lowbit(x);
    }
}
ll kth(ll k,vector<ll>&c){
    ll idx = 0;
    ll bm = 1;
    while((bm<<1)<=MAXN)bm <<= 1;
    for(ll st=bm;st;st>>=1){
        ll nt = idx+st;
        if(nt<=MAXN&&c[nt]<k){
            idx = nt;
            k -= c[nt];
        }
    }
    return idx+1;
}
ll query(ll l,ll r,vector<ll>&c){
    if(l>r) return 0;
    return getsum(r,c)-getsum(l-1,c);
}
//在树状数组上二分查找第k大的树
//只有对统计树状数组的出现频率才能二分，考虑到离散化，所有树状数组都可实现
//多测记得清空
int main(){
    int x;cin>>x;
    for(int i=1;i<=x;i++){
        add(i,1,c);
    }
    int ans = kth(3,c);
    cout<<ans<<endl;
    return 0;
}
```

并查集 19 行

```
const int N = 1e5 + 10;
int n, f[N];
void init()
{
    for (int i = 1; i <= n; i++)
    {
        f[i] = i;
    }
}
int find(int x)
{
    if (x == f[x]) return x;
    return f[x] = find(f[x]);
}
void join(int c1, int c2)
{
    int f1 = find(c1), f2 = find(c2);
    if (f1 != f2) f[f1] = f2;
}
```

普通树状数组

37 行

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
template<typename T>
class BIT {
    vector<T> tree;
public:
    BIT(int n) : tree(n + 1) {}

    void update(int i, T val) {
        for (; i < tree.size(); i += i & -i) tree[i] += val;
    }

    T pre(int i) const {
        T res{};
        for (; i > 0; i &= i - 1) res += tree[i];
        return res;
    }

    T query(int l, int r) const {
        if (r < l) return 0;
        return pre(r) - pre(l - 1);
    }
};
    const int n = 100000;
    BIT<long long> bit(n);      // 下标 1..n
int main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    int i,v,l,r;
    cin>>i>>v>>l>>r;
    bit.update(i, v);           // 位置 i 增加 v（单点加）
    long long s = bit.pre(i);   // 前缀和 a[1]+...+a[i]
    long long t = bit.query(l, r);  // 区间和 a[l]+...+a[r]，l<=r
    return 0;
}
```

随机哈希

35 行

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
mt19937_64 myRand64(chrono::steady_clock::now().time_since_epoch().c
  ount());
void solve()
{
    int n,q;cin>>n>>q;
    int has[125001],pre[250001];
    for(int i=1;i<=125000;i++)has[i]=myRand64();
    for(int i=1;i<=n;i++){
        int a;cin>>a;
        if(a>n/2)pre[i]=pre[i-1];
        else pre[i]=pre[i-1]+has[a];
    }
    for(int i=1;i<=125000;i++)has[i]+=has[i-1]+has[i];
    while(q--){
        int l,r;cin>>l>>r;l--;
        int len = r-l;
        if(!(len&1)&&pre[r]-pre[l]==has[len/2])cout<<"Yes"<<endl;
        else cout<<"No"<<endl;
    }
}
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    int t = 1;
    // cin >> t;
    while (t--)
    {
        solve();
    }
    return 0;
}
```

珂朵莉树

60 行

```cpp
#include <iostream>
#include <map>
using namespace std;

const int MAXI = 1e5;

int q;

map<int, int> mp = {{1, 0}};
map<int, int> cnt = {{0, MAXI + 1}};

void Add(int x, int v) {
    cnt[x] += v;
}

void Remove(int x, int v) {
    auto it = cnt.find(x);
    if (!(it->second -= v)) {
        cnt.erase(it);
    }
}

auto Split(int x) {
    auto it = mp.lower_bound(x);
    if (it->first == x) {
        return it;
    }
    return mp.emplace_hint(it, x, prev(it)->second);
}

void Modify(int l, int r, int x) {
    r++;
    auto itr = Split(r);
    auto itl = Split(l);
    for (auto it = itl; it != itr; it = mp.erase(it)) {
        Remove(it->second, next(it)->first - it->first);
    }
    mp.emplace_hint(itr, l, x);
    Add(x, r - l);
}

void perform(int l, int r) {
    auto itr = Split(r + 1);
    auto itl = Split(l);
    for (auto it = itl; it != itr; ++it) {
        // 对每一个块执行传入的操作
    }
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> q;
    while (q--) {
        int l, r, x;
        cin >> l >> r >> x;
        Modify(l, r, x);
        cout << cnt.size() << '\n';
    }
}
// 64 位输出请用 printf("%lld")
```

区间加乘和单点查询线段树

115 行

```cpp
#include <iostream>
#include <vector>
using namespace std;
//线段树
//区间加乘和单点求值，然后对值进行998244353取模
//op = 1区间加数 op = 2 区间乘数 op = 3 单点查询
using ll = long long;

const int MOD = 998244353;

int n;
int q;
vector<int> a;

struct Node {
    int mul = 1;
    int add = 0;
    void Mul(ll x) {
        mul = (ll)mul * x % MOD;
        add = (ll)add * x % MOD;
    }
    void Add(ll x) {
        add = (add + x) % MOD;
    }
    ll Op(ll x) {
        return (x * mul + add) % MOD;
    }
    void AddTag(const Node& other) {
        mul = (ll)mul * other.mul % MOD;
        add = ((ll)add * other.mul + other.add) % MOD;
    }
};

vector<Node> xds;

void Pushdown(int p) {
    xds[p << 1].AddTag(xds[p]);
    xds[p << 1 | 1].AddTag(xds[p]);
    xds[p] = {};
}

void Add(int l, int r, ll v, int p, int lp, int rp) {
    if (r < lp || rp < l) {return;}
    if (l <= lp && rp <= r) {
        xds[p].Add(v);
        return;
    }
    int mid = (lp + rp) >> 1;
    Pushdown(p);
    Add(l, r, v, p << 1, lp, mid);
    Add(l, r, v, p << 1 | 1, mid + 1, rp);
}

void Mul(int l, int r, ll v, int p, int lp, int rp) {
    if (r < lp || rp < l) {return;}
    if (l <= lp && rp <= r) {
        xds[p].Mul(v);
        return;
    }
    int mid = (lp + rp) >> 1;
    Pushdown(p);
    Mul(l, r, v, p << 1, lp, mid);
    Mul(l, r, v, p << 1 | 1, mid + 1, rp);
}

ll Val(ll x, int p, int l, int r) {
    if (l == r) {
        return xds[p].Op(a[x]);
    }
    int mid = (l + r) >> 1;
    Pushdown(p);
    if (x <= mid) {
        return Val(x, p << 1, l, mid);
    }
    return Val(x, p << 1 | 1, mid + 1, r);
}

void Solve() {
    cin >> n >> q;
    a.resize(n + 1);
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }
    xds.assign(n * 4 + 1, {});
    while (q--) {
        int op;
        cin >> op;
        int l, r;
        ll x;
        switch (op) {
            case 1:
                cin >> l >> r >> x;
                Add(l, r, x, 1, 1, n);
                break;
            case 2:
                cin >> l >> r >> x;
                Mul(l, r, x, 1, 1, n);
                break;
            default:
                cin >> x;
                x = Val(x, 1, 1, n);
                if (x < 0) {
                    x += MOD;
                }
                cout << x << '\n';
                break;
        }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    Solve();
}
```

区间更新线段树

142 行

```cpp
// 模板来源 https://leetcode.cn/circle/discuss/mOr1u6/
#include<bits/stdc++.h>
using namespace std;
template<typename T, typename F>
class LazySegmentTree {
    // 注：也可以去掉 template<typename T, typename F>，改在这里定义 T 和 F
    // using T = pair<int, int>;
    // using F = pair<int, int>;

    // 懒标记初始值
    const F TODO_INIT = 0; // **根据题目修改**

    struct Node {
        T val;
        F todo;
    };

    int n;
    vector<Node> tree;

    // 合并两个 val
    T merge_val(const T& a, const T& b) const {
        return a + b; // **根据题目修改**
    }

    // 合并两个懒标记
    F merge_todo(const F& a, const F& b) const {
        return a + b; // **根据题目修改**
    }

    // 把懒标记作用到 node 子树（本例为区间加）
    void apply(int node, int l, int r, F todo) {
        Node& cur = tree[node];
        // 计算 tree[node] 区间的整体变化
        cur.val += todo * (r - l + 1); // **根据题目修改**
        cur.todo = merge_todo(todo, cur.todo);
    }

    // 把当前节点的懒标记下传给左右儿子
    void spread(int node, int l, int r) {
        Node& cur = tree[node];
        F todo = cur.todo;
        if (todo == TODO_INIT) { // 没有需要下传的信息
            return;
        }
        int m = (l + r) >> 1;
        apply(node * 2, l, m, todo);
        apply(node * 2 + 1, m + 1, r, todo);
        cur.todo = TODO_INIT; // 下传完毕
    }

    // 合并左右儿子的 val 到当前节点的 val
    void maintain(int node) {
        tree[node].val = merge_val(tree[node * 2].val, tree[node * 2 + 1].val);
    }

    // 用 a 初始化线段树
    // 时间复杂度 O(n)
    void build(const vector<T>& a, int node, int l, int r) {
        Node& cur = tree[node];
        cur.todo = TODO_INIT;
        if (l == r) { // 叶子
            cur.val = a[l]; // 初始化叶节点的值
            return;
        }
        int m = (l + r) >> 1;
        build(a, node * 2, l, m); // 初始化左子树
        build(a, node * 2 + 1, m + 1, r); // 初始化右子树
        maintain(node);
    }

    void update(int node, int l, int r, int ql, int qr, F f) {
        if (ql <= l && r <= qr) { // 当前子树完全在 [ql, qr] 内
            apply(node, l, r, f);
            return;
        }
        spread(node, l, r);
        int m = (l + r) >> 1;
        if (ql <= m) { // 更新左子树
            update(node * 2, l, m, ql, qr, f);
        }
        if (qr > m) { // 更新右子树
            update(node * 2 + 1, m + 1, r, ql, qr, f);
        }
        maintain(node);
    }

    T query(int node, int l, int r, int ql, int qr) {
        if (ql <= l && r <= qr) { // 当前子树完全在 [ql, qr] 内
            return tree[node].val;
        }
        spread(node, l, r);
        int m = (l + r) >> 1;
        if (qr <= m) { // [ql, qr] 与右子树无交集，仅需递归左子树
            return query(node * 2, l, m, ql, qr);
        }
        if (ql > m) { // [ql, qr] 与左子树无交集，仅需递归右子树
            return query(node * 2 + 1, m + 1, r, ql, qr);
        }
        // [ql, qr] 与左右子树均有交集，分别递归，然后合并结果
        T l_res = query(node * 2, l, m, ql, qr);
        T r_res = query(node * 2 + 1, m + 1, r, ql, qr);
        return merge_val(l_res, r_res);
    }

public:
    // 线段树维护一个长为 n 的数组（下标从 0 到 n-1），元素初始值为 init_val
    LazySegmentTree(int n, T init_val = 0) : LazySegmentTree(vector<T>(n, init_val)) {}

    // 线段树维护数组 a
    LazySegmentTree(const vector<T>& a) : n(a.size()), tree(2 << bit_width(a.size() - 1)) {
        build(a, 1, 0, n - 1);
    }

    // 用 f 更新 [ql, qr] 中的每个 a[i]
    // 0 <= ql <= qr <= n-1
    // 时间复杂度 O(log n)
    void update(int ql, int qr, F f) {
        update(1, 0, n - 1, ql, qr, f);
    }

    // 返回用 merge_val 合并所有 a[i] 的计算结果，其中 i 在闭区间 [ql, qr] 中
    // 0 <= ql <= qr <= n-1
    // 时间复杂度 O(log n)
    T query(int ql, int qr) {
        return query(1, 0, n - 1, ql, qr);
    }
};

int main() {
    LazySegmentTree<long long, long long> t(8); // 默认值为 0
    t.update(3, 5, 100);
    t.update(4, 6, 10);
    cout << t.query(0, 7) << endl;

    vector<long long> nums = {3, 1, 4, 1, 5, 9, 2, 6};
    LazySegmentTree<long long, long long> t2(nums);
    t2.update(3, 5, 1);
    t2.update(4, 6, 1);
    cout << t2.query(0, 7) << endl;
    return 0;
}
```

无区间更新线段树

109 行

```cpp
// 模板来源 https://leetcode.cn/circle/discuss/mOr1u6/
// 线段树有两个下标，一个是线段树节点的下标，另一个是线段树维护的区间的下标
// 节点的下标：从 1 开始，如果你想改成从 0 开始，需要把左右儿子下标分别改成 node*2+1 和 node*2+2
// 区间的下标：从 0 开始
#include<bits/stdc++.h>
using namespace std;

template<typename T>
class SegmentTree {
    // 注：也可以去掉 template<typename T>，改在这里定义 T
    // using T = pair<int, int>;

    int n;
    vector<T> tree;

    // 合并两个 val
    T merge_val(T a, T b) const {
        return max(a, b); // **根据题目修改**
    }

    // 合并左右儿子的 val 到当前节点的 val
    void maintain(int node) {
        tree[node] = merge_val(tree[node * 2], tree[node * 2 + 1]);
    }

    // 用 a 初始化线段树
    // 时间复杂度 O(n)
    void build(const vector<T>& a, int node, int l, int r) {
        if (l == r) { // 叶子
            tree[node] = a[l]; // 初始化叶节点的值
            return;
        }
        int m = (l + r) >> 1;
        build(a, node * 2, l, m); // 初始化左子树
        build(a, node * 2 + 1, m + 1, r); // 初始化右子树
        maintain(node);
    }

    void update(int node, int l, int r, int i, T val) {
        if (l == r) { // 叶子（到达目标）
            // 如果想直接替换的话，可以写 tree[node] = val
            tree[node] = merge_val(tree[node], val);
            return;
        }
        int m = (l + r) >> 1;
        if (i <= m) { // i 在左子树
            update(node * 2, l, m, i, val);
        } else { // i 在右子树
            update(node * 2 + 1, m + 1, r, i, val);
        }
        maintain(node);
    }

    T query(int node, int l, int r, int ql, int qr) const {
        if (ql <= l && r <= qr) { // 当前子树完全在 [ql, qr] 内
            return tree[node];
        }
        int m = (l + r) >> 1;
        if (qr <= m) { // [ql, qr] 与右子树无交集，仅需递归左子树
            return query(node * 2, l, m, ql, qr);
        }
        if (ql > m) { // [ql, qr] 与左子树无交集，仅需递归右子树
            return query(node * 2 + 1, m + 1, r, ql, qr);
        }
        // [ql, qr] 与左右子树均有交集，分别递归，然后合并结果
        T l_res = query(node * 2, l, m, ql, qr);
        T r_res = query(node * 2 + 1, m + 1, r, ql, qr);
        return merge_val(l_res, r_res);
    }

public:
    // 线段树维护一个长为 n 的数组（下标从 0 到 n-1），元素初始值为 init_val
    SegmentTree(int n, T init_val) : SegmentTree(vector<T>(n, init_val)) {}

    // 线段树维护数组 a
    SegmentTree(const vector<T>& a) : n(a.size()), tree(2 << bit_width(a.size() - 1)) {
        build(a, 1, 0, n - 1);
    }

    // 更新 a[i] 为 merge_val(a[i], val)
    // 时间复杂度 O(log n)
    void update(int i, T val) {
        update(1, 0, n - 1, i, val);
    }

    // 返回用 merge_val 合并所有 a[i] 的计算结果，其中 i 在闭区间 [ql, qr] 中
    // 时间复杂度 O(log n)
    T query(int ql, int qr) const {
        return query(1, 0, n - 1, ql, qr);
    }

    // 获取 a[i] 的值
    // 时间复杂度 O(log n)
    T get(int i) const {
        return query(1, 0, n - 1, i, i);
    }
};

int main() {
    SegmentTree t(8, 0LL); // 如果这里写 0LL，那么 SegmentTree 存储的就是 long long 数据
    t.update(0, 1LL << 60);
    cout << t.query(0, 7) << endl;

    vector<int> nums = {3, 1, 4, 1, 5, 9, 2, 6};
    // 注意：如果要让 SegmentTree 存储 long long 数据，需要传入 vector<long long>
    SegmentTree t2(nums); // 这里 SegmentTree 存储的是 int 数据
    cout << t2.query(0, 7) << endl;
    return 0;
}
```

树状数组异或求和

128 行

```cpp
#include <array>
#include <iostream>
#include <valarray>
#include <vector>
//异或线段数板子 输入1 l r 代表对l r区间求和 输入2 l r k代表区间内的所有数都异或于k
using namespace std;

using ll = long long;

const int WEI = 17;

struct Node {
    int tag = 0;
    array<int, WEI> cnt;
    Node() = default;
    Node(int a) {
        for (int i = 0; i < WEI; i++) {
            cnt[i] = (a >> i) & 1;
        }
    }
    void AddTag(int x, int l, int r) {
        int len = r - l + 1;
        for (int i = 0; i < WEI; i++) {
            if (x & (1 << i)) {
                cnt[i] = len - cnt[i];
            }
        }
        tag ^= x;
    }
    ll Sum() {
        ll res = 0;
        for (int i = 0; i < WEI; i++) {
            res += (ll)cnt[i] << i;
        }
        return res;
    }
    void Update(const Node& a, const Node& b) {
        for (int i = 0; i < WEI; i++) {
            cnt[i] = a.cnt[i] + b.cnt[i];
        }
    }
};

int n;
int m;

vector<int> a;
vector<Node> xds;

void Pushdown(int p, int l, int r) {
    if (xds[p].tag) {
        int mid = (l + r) >> 1;
        xds[p << 1].AddTag(xds[p].tag, l, mid);
        xds[p << 1 | 1].AddTag(xds[p].tag, mid + 1, r);
        xds[p].tag = 0;
    }
}

void Update(int p) {
    xds[p].Update(xds[p << 1], xds[p << 1 | 1]);
}

void Build(int p, int l, int r) {
    if (l == r) {
        xds[p] = a[l];
        return;
    }
    int mid = (l + r) >> 1;
    Build(p << 1, l, mid);
    Build(p << 1 | 1, mid + 1, r);
    Update(p);
}

void Xor(int l, int r, int k, int p, int lp, int rp) {
    if (r < lp || rp < l) {
        return;
    }
    if (l <= lp && rp <= r) {
        xds[p].AddTag(k, lp, rp);
        return;
    }
    Pushdown(p, lp, rp);
    int mid = (lp + rp) >> 1;
    Xor(l, r, k, p << 1, lp, mid);
    Xor(l, r, k, p << 1 | 1, mid + 1, rp);
    Update(p);
}

ll Sum(int l, int r, int p, int lp, int rp) {
    if (r < lp || rp < l) {
        return 0;
    }
    if (l <= lp && rp <= r) {
        return xds[p].Sum();
    }
    Pushdown(p, lp, rp);
    int mid = (lp + rp) >> 1;
    return Sum(l, r, p << 1, lp, mid) + Sum(l, r, p << 1 | 1, mid + 1, rp);
}

void Solve() {
    cin >> n >> m;
    a.resize(n + 1);
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }
    xds.resize(n * 4 + 1);
    Build(1, 1, n);
    while (m--) {
        int op, l, r;
        cin >> op >> l >> r;
        switch (op) {
            case 1:
                cout << Sum(l, r, 1, 1, n) << '\n';
                break;
            case 2:
                int k;
                cin >> k;
                Xor(l, r, k, 1, 1, n);
        }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    Solve();
}
```

数学

18 个模板

gcd求最大公因数

15 行

```cpp
#include<bits/stdc++.h>
using namespace std;
int gcd(int a,int b){
    while(b){
        int t = b;
        b = a%b;
        a = t;
    }
    return a;
}
int main(){
    int a,b;cin>>a>>b;
    cout<<gcd(a,b)<<endl;
    return 0;
}
```

余数求和

25 行

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
void solve(){
    ll n,k;cin>>n>>k;
    ll ans  = 0;
    for(ll l =1,r;l<=n;l = r+1){
        ll t = k/l;
        if(t>0)r = min(n,k/t);
        else r = n;
        ans += k*(r-l+1)-(r-l+1)*(l+r)/2*t;
    }
    cout<<ans<<endl;
}
int main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    int t = 1;
    //cin >> t;
    while (t--){
        solve();
    }
    return 0;
}
```

分解质因数

33 行

```cpp
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
const int MAXN = 1e6 + 5;
vector<int> p;
bool vis[MAXN];

void sieve(int n = MAXN - 1) {
    for (int i = 2; i <= n; ++i) {
        if (!vis[i]) p.push_back(i);
        for (int x : p) {
            if (1LL * i * x > n) break;
            vis[i * x] = true;
            if (i % x == 0) break;
        }
    }
}
vector<pair<ll, int>> ft(ll n) {
    vector<pair<ll, int>> res;
    for (int x : p) {
        if (1LL * x * x > n) break;
        if (n % x == 0) {
            int cnt = 0;
            while (n % x == 0) n /= x, ++cnt;
            res.emplace_back(p, cnt);
        }
    }
    if (n > 1) res.emplace_back(n, 1);
    return res;
}
int main(){

}
```

容斥原理算区间互质数量

52 行

```cpp
#include<bits/stdc++.h>
using namespace std;
// 质因数分解
vector<int> to_ys(int tar){
    vector<int> ans;
    for(int i=2;i*i<=tar;i++){
        if(tar%i==0){
            ans.push_back(i);
            while(tar%i==0)tar/=i;
        }
    }
    if(tar>1)ans.push_back(tar);
    return ans;
}
//计算有多少个1 r中的数与n互质
int to_ct(int r,int n){
    vector<int> ys = to_ys(n);
    int len = r;
    int yss = ys.size();
    for(int i=1;i<(1<<yss);i++){
        int ct = 0,tp = 1;
        bool out = false;
        for(int j=0;j<yss;j++){
            if(i & (1 << j)){
                if(tp>r/ys[j]){out=true;break;}
                tp*=ys[j];
                ct++;}
            }
        if(out)continue;
        if(ct%2==1)len-=(r/tp);else len+=(r/tp);
    }
    return len;
}
//计算有多少个l r中的数与n互质
int to_res(int l,int r,int n){
    int ans = to_ct(r,n);
    if(l>1)ans-=to_ct(l-1,n);
    return ans;
}
int main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    int T;cin>>T;
    while(T--){
        int l,r,n;cin>>l>>r>>n;
        int zhi = to_res(l,r,n);
        int len = r-l+1;
        int ans = (len-zhi)*2+zhi;
        cout<<ans<<endl;
    }
    return 0;
}
```

快速幂

18 行

```cpp
#include <bits/stdc++.h>
using namespace std;

int fp(int a,int b) {
    int res = 1;
    while (b>0) {
        if (b%2==1) res=res*a;
        a=a*a;
        b/=2;
    }
    return res;
}
int main(){
    int a,b;cin>>a>>b;
    int ans = fp(a,b);
    cout<<ans<<endl;
    return 0;
}
```

按位异或和

12 行

```cpp
#include<bits/stdc++.h>
using namespace std;
//1到n的按位异或和
long long get_val(long long n){
    switch(n%4){
        case 0: return n;
        case 1: return 1;
        case 2: return n+1;
        case 3: return 0;
    }
    return -1;
}
```

杨辉三角

42 行

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
void solve(){
    int n;cin>>n;
    vector<vector<int>> a = {{},{1},{1,1}};
    for(int i=3;i<=n;i++){
        vector<int> b(i,1);
        for(int j=1;j<i-1;j++)b[j] = a[i-1][j]+a[i-1][j-1];
        a.push_back(b);
    }
    // for(int i=1;i<=n;i++){
    //     for(int j=0;j<i;j++)cout<<a[i][j]<<' ';
    //     cout<<endl;
    // }
    //杨辉三角的点为C(i,n-1)//n是当前层数
    int t,c;cin>>t>>c;//这个逻辑仅用于求组合数
    c--;
    int ans = 1;
    t = min(c-1-t,t);
    cout<<t<<' '<<c+1-t<<' '<<c<<endl;
    for(int i=c-t+1;i<=c;i++)ans*=i;

    for(int i=1;i<=t;i++)ans/=i;
    cout<<ans<<endl;
    cout<<a[c+1][t]<<endl;
    //测试输入
    // 2 5 6
    // 15
    // 15
}
int main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    int t = 1;
    //cin >> t;
    while (t--){
        solve();
    }
    return 0;
}
```

构造格雷码

24 行

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
int g(int n){return n^(n>>1);}
int rev_g(int g) {
  int n = 0;
  for (; g; g >>= 1) n ^= g;
  return n;
}
void solve(){
    int n;cin>>n;
    for(int i=0;i<n;i++)cout<<rev_g(g(i))<<endl;
}
int main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    int t = 1;
    //cin >> t;
    while (t--){
        solve();
    }
    return 0;
}
```

模逆元

26 行

```cpp
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
const int mod = 1e9+7;//注意！！！使用题目给的模逆元
//通过费马小定理计算模逆元（需要
//原理 a作为除数，模逆元等于a^(mod-2)%mod
//理由 计算 a 的 (mod-1) 次方，然后除以 mod，得到的余数一定等于 1。
  所以a的mod-2次方就是a作为除数的时的模逆元
ll fp(ll a,ll b,ll mod) {
    ll res = 1;
    while (b>0) {
        if (b%2==1) res=res*a%mod;
        a=a*a%mod;
        b/=2;
    }
    return res;
}
int main(){
    ll a,b;
    cin>>a>>b;
    //b作为除数
    ll invb = fp(b,mod-2,mod);// invb就是模逆元
    //模逆元使用方法
    //(A / B) % MOD = (A % MOD) * inv(B) % MOD
    ll ans = (a%mod)*(invb)%mod;
    return 0;
}
```

欧拉函数快速幂

25 行

```cpp
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
const ll MOD = 1e9+7;
//注意到2023由7*17*17组成
//求1到2023的2023次方中有多少个整数与2023互质
ll ct_pw(ll x,ll y){
    ll ans = 1;
    while(y>0){
        if(y%2==1)ans=ans*x%MOD;
        x=x*x%MOD;
        y/=2;
    }
    return ans;
}
int main(){
    ll t,a,b,c;
    t = ct_pw(2023,2023);
    a = ct_pw(7,2022)*ct_pw(289,2023)%MOD;
    b = ct_pw(17,2022)*ct_pw(119,2023)%MOD;
    c = ct_pw(119,2022)*ct_pw(17,2023)%MOD;
    ll ans = (t-a-b+c)%MOD;
    cout<<ans<<endl;
    return 0;
}
```

求二进制的1位数

19 行

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
void solve(){
    int x,y;cin>>y;
    x = __builtin_popcount(y);

}
int main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    int t = 1;
    //cin >> t;
    while (t--){
        solve();
    }
    return 0;
}
```

矩阵快速幂

55 行

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll MOD =  1e9+7;
//用0结尾的序列可以连接用1开头的序列和用0开头的序列
//用1结尾的序列可以连接用1开头的序列
void addi(ll& a,ll& b){
    a = (a+b)%MOD;
}
ll n;
string s;
vector<vector<ll>> dp;
//矩阵相乘
vector<vector<ll>> mul(const vector<vector<ll>>& a,const vector<vect
  or<ll>>& b){
    vector<vector<ll>> res = {{1,0},{0,1}};
    for(ll i=0;i<2;i++)for(ll j=0;j<2;j++){
        ll cur = 0;
        for(ll k=0;k<2;k++)cur+=(ll)(a[i][k]*b[k][j]);
        res[i][j] = cur%MOD;
    }
    return res;
}
void solve(){
    ll n;cin>>n;
    string s;cin>>s;
    dp = {{1,0},{0,1}};
    for(const auto& c:s){
        if(c=='0')addi(dp[0][0],dp[0][0]);
        else{
            addi(dp[1][0],dp[1][0]);
            addi(dp[1][0],dp[0][0]);
            addi(dp[1][1],dp[1][1]);
        }
    }
    //快速幂
    vector<vector<ll>> res = {{1,0},{0,1}};
    while(n){
        if(n&1)res = mul(res,dp);
        dp = mul(dp,dp);
        n >>= 1;
    }
    cout<<(res[0][0]+res[1][0]+(ll)MOD-1)%MOD<<endl;

}
int main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    ll t = 1;
    cin >> t;
    while (t--){
        solve();
    }
    return 0;
}
```

线性基求最大异或和之和

36 行

```
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
//将一个整数列表分为两个集合使这两个集合的异或和的和最大，输出最大得分
vector<ll> b(61,0);
void insert(ll x){
    for(ll i=60;i>=0;i--){
        if(((x >> i)&1)==0)continue;
        if(b[i]==0){b[i]=x;return ;}
        x^=b[i];
    }
}
ll to_res(ll n,vector<ll> a){
    fill(b.begin(),b.end(),0);
    ll tol = 0;
    for(ll i=0;i<n;i++)tol^=a[i];
    ll mask = ((1LL << (60 + 1)) - 1)^tol;
    for(ll i=0;i<n;i++)insert(a[i] & mask);
    ll mx = 0;
    for(ll i=60;i>=0;i--){
        if((mx^b[i])>mx)mx^=b[i];
    }
    return tol+mx*2;

}
int main(){
    ll T;cin>>T;
    while(T--){
        ll n;cin>>n;
        vector<ll> a(n);
        for(ll i=0;i<n;i++)cin>>a[i];
        ll ans = to_res(n,a);
        cout<<ans<<endl;
    }
    return 0;
}
```

组合数

39 行

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const int MOD = 1e9+7;
const int MAXI = 1e6-2;
array<int,MAXI+1> fac;
array<int,MAXI+1> invFac;
ll n,m;
//求C上(n-2)下(m-1);
void init(){
    invFac[1] = 1;
    for(int i=2;i<=MAXI;i++)
        invFac[i] = (ll)(MOD-MOD/i)*invFac[MOD%i]%MOD;
        fac[0] = 1;
        invFac[0] = 1;
        for(int i=1;i<=MAXI;i++){
            fac[i] = (ll)fac[i-1]*i%MOD;
            invFac[i] = (ll)invFac[i-1]*invFac[i]%MOD;
        }
}
ll to_C(ll x,ll y){
    return (ll)fac[x]*invFac[x-y]%MOD*invFac[y]%MOD;
}
void solve(){
    cin>>n>>m;
    cout<<to_C(n-2,m-1)<<endl;
}
int main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    init();
    int t = 1;
    cin >> t;
    while (t--){
        solve();
    }
    return 0;
}
```

质数筛

22 行

```
#include<bits/stdc++.h>
using namespace std;
const int TOP = 1e6+1;
vector<int> ss(){
    vector<int> ans;
    vector<bool> bl(TOP,true);
    for(int i=2;i<TOP;i++){
        if(bl[i]){
            ans.push_back(i);
            if(i>TOP/i)continue;
            for(int j=i*i;j<TOP;j+=i)bl[j]=false;
        }
    }
    return ans;
}
int main(){
    int n;cin>>n;
    vector<int> nums = ss();
    int len = nums.size();
    for(int i=0;i<len;i++)cout<<nums[i]<<' ';
    return 0;
}
```

输出int128

13 行

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
void print(__int128 x) {
    if (x < 0) {
    putchar('-');
    x = -x;
}
if (x > 9) {
    print(x / 10);
}
    putchar(x % 10 + '0');
}
```

进制转化

22 行

```
#include<bits/stdc++.h>
using namespace std;
//十进制转换二进制字符串
string to_b(int x){
    string ans;
    while(x>0){
        ans.push_back((char)(x%2+'0'));
        x/=2;
    }
    reverse(ans.begin(),ans.end());
    return ans;
}
//二进制字符串转换十进制
int to_v(string s){
    if(s.empty()||s=="0")return 0;
    int tp = 1,ans = 0;
    for(int i=(int)s.length()-1;i>=0;i--){
        if(s[i]=='1')ans+=tp;
        tp*=2;
    }
    return ans;
}
```

大数加减乘除

116 行

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

bool lt(const string&x,const string&y){
    if(x.size()!=y.size())return x.size()<y.size();
    return x<y;
}

string str_sum(string s1,string s2){
    int i = s1.length()-1;
    int j = s2.length()-1;
    int carry = 0;
    string ans;
    while(i>=0||j>=0||carry>0){
        int sum = carry;
        if(i>=0)sum+=(s1[i--]-'0');
        if(j>=0)sum+=(s2[j--]-'0');
        ans.push_back((sum%10)+'0');
        carry = sum/10;
    }
    reverse(ans.begin(),ans.end());
    return ans;
}

string str_sub(string a, string b) {
    bool neg = false;
    if (lt(a, b)) { swap(a, b); neg = true; }
    int i = a.size() - 1, j = b.size() - 1;
    int borrow = 0;
    string ans = "";
    while (i >= 0 || j >= 0) {
        int diff = -borrow;
        if (i >= 0) diff += (a[i--] - '0');
        if (j >= 0) diff -= (b[j--] - '0');
        if (diff < 0) { diff += 10; borrow = 1; } else { borrow = 0;
   }
        ans.push_back(diff + '0');
    }
    while (ans.size() > 1 && ans.back() == '0') ans.pop_back();
    if (neg) ans.push_back('-');
    reverse(ans.begin(), ans.end());
    return ans;
}

string str_mul(string a, string b) {
    if (a == "0" || b == "0") return "0";
    int n = a.size(), m = b.size();
    vector<int> res(n + m, 0);
    for (int i = n - 1; i >= 0; --i) {
        for (int j = m - 1; j >= 0; --j) {
            int mul = (a[i] - '0') * (b[j] - '0');
            int sum = mul + res[i + j + 1];
            res[i + j + 1] = sum % 10;
            res[i + j] += sum / 10;
        }
    }
    string ans = "";
    for (int num : res) {
        if (!(ans.empty() && num == 0)) ans.push_back(num + '0');
    }
    return ans.empty() ? "0" : ans;
}

string str_mul_single(string a, int b) {
    if (a == "0" || b == 0) return "0";
    int carry = 0;
    string ans = "";
    for (int i = a.size() - 1; i >= 0; --i) {
        int prod = (a[i] - '0') * b + carry;
        ans.push_back((prod % 10) + '0');
        carry = prod / 10;
    }
    while (carry) { ans.push_back((carry % 10) + '0'); carry /= 10;
  }
    reverse(ans.begin(), ans.end());
    return ans;
}

string str_div(string a, string b) {
    if (b == "0") return "ERROR";
    if (lt(a, b)) return "0";
    string rem = "";
    string ans = "";
    for (char c : a) {
        rem.push_back(c);
        size_t pos = rem.find_first_not_of('0');
        if (pos != string::npos) rem = rem.substr(pos); else rem = "
  0";
        int q = 0;
        while (q < 9) {
            string tmp = str_mul_single(b, q + 1);
            if (!lt(rem, tmp)) q++; else break;
        }
        ans.push_back(q + '0');
        string sub_val = str_mul_single(b, q);
        rem = str_sub(rem, sub_val);
    }
    size_t pos = ans.find_first_not_of('0');
    if (pos != string::npos) ans = ans.substr(pos); else ans = "0";
    return ans;
}

void solve(){
    string a,b;cin>>a>>b;
    cout<<str_sum(a,b)<<" "<<str_sub(a,b)<<" "<<str_mul(a,b)<<" "<<s
  tr_div(a,b)<<"\n";
}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    int t = 1;
    cin >> t;
    while (t--){
        solve();
    }
    return 0;
}
```

几何 2 个模板

叉乘公式 36 行

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
//差乘判断是否相交
//+1代表线段是否从左到右，-1代表是否从右到左
double a,b,c,d;
double u1,v1,u2,v2;
double cross(double x1,double y1,double x2,double y2,double x3,doubl
  e y3){
    return (y3-y1)*(x2-x1)-(x3-x1)*(y2-y1);
}
void solve(){
    double n,ans = 0;cin>>n;
    cin>>u1>>v1>>u2>>v2;
    for(int i=0;i<n;i++){
        cin>>a>>b>>c>>d;
        double s1 = cross(u1,v1,u2,v2,a,b);
        double s2 = cross(u1,v1,u2,v2,c,d);
        double t1 = cross(a,b,c,d,u1,v1);
        double t2 = cross(a,b,c,d,u2,v2);
        if(s1*s2<0&&t1*t2<0){
            if(s1>0)ans++;else ans--;
        }
    }
    cout<<ans<<endl;
}
int main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    int t = 1;
    //cin >> t;
    while (t--){
        solve();
    }
    return 0;
}
```

线段在圆中的长度函数 55 行

```
#include<bits/stdc++.h>
using namespace std;
typedef double db;
const db eps = 1e-12;
vector<db> to_mid(db x0,db y0,db rr,db ax,db ay,db bx,db by){
    db dx = bx - ax,dy = by - ay;
    db a = dx*dx+dy*dy;
    db b = dx*(ax-x0)+dy*(ay-y0);
    db c = (ax-x0)*(ax-x0)+(ay-y0)*(ay-y0)-rr*rr;
    db dt = b*b-a*c;
    if(dt<0)return {0.0,-1.0};
    db sum = sqrt(dt);
    db t1 = (-b-sum)/a,t2 = (-b+sum)/a;
    if(t1>t2)swap(t1,t2);
    db l = max(t1,0.0),r = min(t2,1.0);
    if(l>r){
        if(l-r<eps){
            db t = (l+r)/2.0;
            return {t,t};
        }
        return {0.0,-1.0};
    }
    return {l,r};
}
int main(){
    db x1,y1,r1,x2,y2,r2;
    db a,b,c,d;
    cin>>x1>>y1>>r1;
    cin>>x2>>y2>>r2;
    cin>>a>>b>>c>>d;
    db dx = c-a,dy = d-b;
    db xd_len = hypot(dx,dy);
    if(xd_len<eps){cout<<0.0<<endl;return 0;}
    vector<db> mid1 = to_mid(x1,y1,r1,a,b,c,d);
    vector<db> mid2 = to_mid(x2,y2,r2,a,b,c,d);
    vector<vector<db>> mids;
    if(mid1[0]<=mid1[1])mids.push_back(mid1);
    if(mid2[0]<=mid2[1])mids.push_back(mid2);
    if(mids.empty()){cout<<0.0<<endl;return 0;}
    sort(mids.begin(),mids.end());
    db t = 0.0;
    db l = mids[0][0],r = mids[0][1];
    if(mids.size()==2){
        if(mids[1][0]<=r+eps)r = max(r,mids[1][1]);
        else {
            t += r-l;
            l = mids[1][0];
            r = mids[1][1];
        }
    }
    t+=r-l;
    db ans = t*xd_len;
    printf("%.10f",ans);
    return 0;
}
```

模拟 1 个模板

德州扑克 76 行

```
#include<bits/stdc++.h>
#include<unordered_map>
using namespace std;
typedef long long ll;
pair<int,int> to_p(string s){
    int sum = -1,pos = -1;
    switch(s[0]){
        case 'A': pos = 13;break;
        case 'J': pos = 10;break;
        case 'Q': pos = 11;break;
        case 'K': pos = 12;break;
        case 'T': pos = 9;break;
        default: break;
    }
    if(s[0]>'1'&&s[0]<='9')pos = (int)(s[0]-'0'-1);
    switch(s[1]){
        case 'C': sum = 1;break;
        case 'D': sum = 2;break;
        case 'H': sum = 3;break;
        case 'S': sum = 4;break;
    }
    return {pos,sum};
}
pair<int,ll> to_pos(vector<pair<int,int>> a){
    sort(a.begin(),a.end());
    bool s_c = true;
    for(int i=0;i<4;i++){if(a[i].second!=a[i+1].second)s_c=false;}
    vector<int> mins = {1,2,3,4,13};bool min_st = true;
    for(int i=0;i<5;i++)if(a[i].first!=mins[i])min_st=false;
    bool s_s = min_st;
    if(!s_s){s_s = true;for(int i=0;i<4;i++){if(a[i].first!=a[i+1].f
  irst-1)s_s=false;}}
    if(s_s)return {s_c?9:5,min_st?0:(ll)a[4].first};
    unordered_map<int,int> mp;
    for(int i=0;i<5;i++)mp[a[i].first]++;
    vector<pair<int,int>> g;
    for(const auto&pair:mp)g.push_back({pair.first,pair.second});
    sort(g.begin(),g.end(),[](pair<int,int>x,pair<int,int>y){
        if(x.second!=y.second)return x.second>y.second;
        return x.first>y.first;
    });
    ll key = 0;
    for(const auto&x:g)key = key*100+x.first;
    if(g[0].second==4)return {8,key};
    if(g[0].second==3&&g[1].second==2)return {7,key};
    if(s_c)return {6,key};
    if(g[0].second==3)return {4,key};
    if(g[0].second==2&&g[1].second==2)return {3,key};
    if(g[0].second==2)return {2,key};
    return {1,key};
}
int cmp_hand(vector<pair<int,int>> x,vector<pair<int,int>> y){
    pair<int,ll> a = to_pos(x),b = to_pos(y);
    if(a<b)return -1;
    if(a>b)return 1;
    return 0;
}
void solve(){
    vector<pair<int,int>> a,b;
    for(int i=0;i<5;i++){string s;cin>>s;a.push_back(to_p(s));}
    for(int i=0;i<5;i++){string s;cin>>s;b.push_back(to_p(s));}
    int res = cmp_hand(a,b);
    if(res>0)cout<<"1"<<endl;
    else if(res<0)cout<<"-1"<<endl;
    else cout<<"0"<<endl;
}
int main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    int t = 1;
    // cin >> t;
    while (t--){
        solve();
    }
    return 0;
}
```

对拍 1 个模板

生成随机数 12 行

```
#include<bits/stdc++.h>
#include<sys/timeb.h>
int main()
{
    struct _timeb T;
    _ftime(&T);
    srand(T.millitm);
    //生成随机数种子，利用 timeb 生成毫秒级别随机数

    printf("%d %d\n", rand(), rand());
    //这样就生成了2个随机数
}
```